

Feasibility Report

Project: Centralized University Portal & Placement Management System

Feasibility Analysis

Technical Feasibility

Technology Stack: The system will utilize a robust, hybrid-cloud Three-Tier Architecture to ensure scalability and real-time performance. The frontend will be built using React.js/Next.js for a responsive, Single Page Application (SPA) experience. The backend will rely on Node.js/Express to handle stateless REST API requests and business logic. A polyglot persistence strategy will be employed: PostgreSQL will serve as the primary relational database for ACID-compliant academic and transactional records, while Firebase will handle real-time states and identity management.

Integration & Tools: Firebase Authentication will be used for secure SSO and Multi-Factor Authentication (MFA), while Firebase Cloud Storage will handle student resume (PDF) uploads via pre-signed URLs. Firebase Cloud Messaging (FCM) will replace legacy polling with instant push notifications for interview alerts. External APIs like SendGrid or Twilio will serve as email/SMS fallbacks.

Development Resources: The chosen stack (React/Node/PostgreSQL/Firebase) is heavily supported by extensive documentation and open-source tooling, reducing the technical learning curve. Technical risks, such as handling the "thundering herd" problem during mass registration, will be mitigated through connection pooling and stateless JWT verification.

Operational Feasibility

Stakeholder Adoption: The portal is designed with role-specific dashboards (Student, TPO/Admin, Recruiter, Faculty) to ensure high usability and direct relevance to each user type. By replacing fragmented WhatsApp groups and manual Excel sheets with an integrated mobile-first UI, student buy-in is expected to be immediate.

Fit with Current Processes: The system serves as a Single Source of Truth (SSOT), mapping directly to the university's Permanent Registration Number (PRN) system. It automates existing manual tasks—such as checking a student's CGPA against a company's requirements—without forcing a disruptive change to standard academic workflows.

Legal & Ethical Feasibility

Data Privacy & Compliance: The system handles sensitive Personally Identifiable Information (PII), including academic transcripts and resumes. The architecture complies with data

protection standards by implementing Role-Based Access Control (RBAC); for example, students cannot view other students' CGPAs, and recruiters only see profiles of students who meet their eligibility thresholds.

Security Measures: All PII will be encrypted at rest and in transit (AES-256 and TLS 1.3). Passwords and identities are managed securely off-server via Firebase Auth, ensuring institutional liability regarding credential storage is minimized.

Ethical Considerations: The "Intelligent Eligibility Engine" ensures objective, automated shortlisting based strictly on quantitative metrics (e.g., CGPA > 8.0, no active backlogs), removing human bias from the initial screening phase.

Schedule Feasibility

Phased Timeline: The project will be executed using an iterative Agile SDLC.

- **Sprint Zero:** Setup of Firebase environments, PostgreSQL schemas, and CI/CD pipelines.
- **Phase 1:** Authentication, RBAC integration, and Student Profile generation.
- **Phase 2:** Recruiter job postings and the automated Eligibility Engine.
- **Phase 3:** Quiz Designer module and Interview live-tracking.
- **Phase 4:** TPO Analytics dashboard and final FCM notification integrations.

Dependencies & Buffer: API integrations (Firebase, SendGrid) and load testing for concurrent users will require a 10-15% schedule buffer.

Functional Requirements

Authentication & User Management:

1. The system shall authenticate all users via Firebase Auth (SSO, Google Auth, or Email/Password) and issue stateless JWTs for session management.
2. The system shall enforce strict Role-Based Access Control (RBAC) to distinguish between Student, TPO/Admin, Faculty, and Recruiter privileges.
3. The system shall allow the TPO to manually override roles and approve/reject new recruiter registrations.

Student Module:

4. The system shall allow students to generate profiles tied securely to their university PRN, displaying their branch, CGPA, and skills.
5. The system shall permit students to upload PDF resumes to Firebase Cloud Storage (capped at 5MB).
6. The system shall provide a real-time feed of eligible job postings and allow one-click applications.

7. The system shall instantly notify students of shortlisting or interview schedules via FCM push notifications.

Faculty Module:

8. The system shall allow faculty members to view department-specific placement statistics and analytics.

9. The system shall allow faculty to endorse students or post departmental notices to student dashboards.

Company/Recruiter Module:

10. The system shall enable recruiters to post detailed roles, specifying CTC and hard eligibility criteria (e.g., branch, minimum CGPA).

11. The system shall execute an Intelligent Eligibility Engine to automatically filter and shortlist applicants who meet the exact job criteria, directly from the PostgreSQL database.

12. The system shall provide an integrated "Quiz Designer" for recruiters to deploy preliminary technical MCQs.

13. The system shall allow companies to update student tracking states (e.g., Applied $\rightarrow$ Interview $\rightarrow$ Hired) in real-time.

Data Management:

14. The system shall store all relational data (jobs, applications, academic records) in a structured PostgreSQL database.

15. The system shall allow TPO administrators to generate and export placement metrics (by CTC, department, or year) in CSV or PDF formats.

Non-Functional Requirements

Performance:

- The system shall load primary UI dashboards in under 3 seconds.
- The API gateway shall respond to job filtering queries in under 200ms.
- The infrastructure shall support 1,000+ concurrent user sessions during peak "Day 0" hiring events without service degradation.

Security:

- All data in transit shall be secured via HTTPS/TLS 1.3, and PII at rest in PostgreSQL shall be encrypted via AES-256.

- Firebase Admin SDK shall be used as middleware to decode and verify user identity tokens securely, preventing unauthorized API access.

Usability:

- The frontend shall utilize responsive design principles (Tailwind CSS) to ensure seamless operation across mobile devices, tablets, and desktop displays.
- The interface shall be highly intuitive, allowing first-time students and recruiters to navigate applications without extensive manual training.

Reliability & Availability:

- The platform shall ensure 99.95% availability during the critical placement season (August–May).
- The system shall implement automated, daily off-site backups of the PostgreSQL database at off-peak hours (e.g., 02:00 IST).

Maintainability:

- The codebase shall be fully decoupled into a Micro-Service Hybrid Architecture (separated Next.js frontend and Node.js backend).
- All backend REST APIs shall be comprehensively documented using Swagger (OpenAPI 3.0) for future developer hand-offs.

Portability & Compatibility:

- The system shall be containerized using Docker, ensuring that the application can be seamlessly deployed to AWS, Google Cloud Platform, or local university Linux servers without environment conflicts.
- The portal shall function optimally on all modern web browsers (Chrome, Firefox, Safari, Edge) without requiring proprietary plugins.

Summary

These non-functional and functional requirements define a modern, highly secure, and highly scalable ecosystem. By upgrading to a hybrid cloud architecture powered by PostgreSQL and Firebase, the Centralized University Portal will successfully transition the University of Hyderabad from a disjointed manual process into a fast, data-driven environment, ensuring that the final application is robust enough to meet the institution's long-term placement needs.